

图片召回实验的数据获取方案

面向本地 MacBook 的多模态 RAG 图片召回子模块

1. 结论

你说得对，上一版没有把“数据怎么来”讲清楚。这个子模块的第一阶段不需要大量数据；目标不是训练模型，而是验证图片召回、元信息融合、API 服务和后续 Agent 调用链路。建议先准备 300-800 张图片就够了。最省事的方案是把数据获取也做成脚本：从公开 Hugging Face 数据源抽样，统一保存为本地 ImageFolder 风格目录，并生成 metadata.jsonl。检索模块只读本地目录，不关心图片原来来自哪里。

2. 推荐的最小数据组合

用途	推荐数据源	建议抽样	为什么适合
硬件/产品图	Doshiba/pcpartpicker-parts-dataset	CPU/GPU/硬盘各 30-60 张	有 image_url、品牌、名称、规格、价格等字段，正好测试图片 + 元信息混合检索。
文档模板图	vaclavpechtor/rvl_cdip-small-200	每类 10-20 张	扫描文档模板很像，适合验证“视觉相似但内容需要靠元信息区分”的问题。
收据/发票	Voxel51/consolidated_receipt_data set	可选 50-100 张	有 OCR 标注和结构化字段，适合后续测试金额、商户、subtotal 等字段检索。
专利/技术图	danaaubakirova/patfig	可选，不默认下载	有专利图、caption、publication number 等元信息，但体积更大且非商业许可。

第一版建议默认只跑硬件数据；如果还想覆盖文档模板相似的问题，再加 RVL-CDIP small。收据和专利可以作为第二阶段可选项。

3. 为什么不需要“大量数据”

这里要验证的是一个召回子系统：图片 embedding 是否能建索引，文本/元信息是否能补充召回，API 是否能被 Agent 调用，MacBook 上速度是否可接受。300 张可以验证功能，1000-3000 张可以验证基本性能；更大的数据只是在后续测试规模化时才需要。

因此不建议一开始下载 LAION、CC12M、Open Images 或机器人视频数据集。这些数据源很大，预处理复杂，而且对当前目标贡献不大。

4. 数据目录契约

所有数据最后都落成本地普通文件夹。这样现有图片检索服务可以完全不依赖 Hugging Face，也方便你未来替换成自己的业务图片。

```
data/corpus/  
  images/  
    pcpartpicker/cpu/493350.jpg  
    pcpartpicker/video-card/123456.jpg  
    pcpartpicker/internal-hard-drive/987654.jpg  
    rvl_cdip/email/000001.jpg  
    rvl_cdip/letter/000002.jpg  
  metadata.jsonl
```

metadata.jsonl 每行对应一张图片，包含 file_name、dataset、category、title、brand、source_url、image_url、text 和 metadata。检索索引时，把 text 字段和 sidecar JSON/TXT 一起写入 SQLite FTS。

5. 默认复现命令

```
# 1. 拉一个很小的硬件样本集  
python scripts/bootstrap_data.py hardware --out data/corpus --per-category 30  
  
# 2. 可选：拉文档模板样本  
python scripts/bootstrap_data.py documents --out data/corpus --per-label 10  
  
# 3. 建索引  
python -m app.cli index --root data/corpus/images --metadata data/corpus/metadata.jsonl  
  
# 4. 启动 API  
uvicorn app.api:app --host 127.0.0.1 --port 8000 --reload  
  
# 5. 冒烟评测  
python scripts/evaluate_smoke.py --queries data/queries/smoke_queries.jsonl --top-k 10
```

6. bootstrap_data.py 应该做什么

这个脚本就是“帮用户准备实验图片”。它不属于在线服务路径，只是本地初始化工具。

1. 读取公开数据源。硬件数据用 Hugging Face datasets streaming 读取 PCPartPicker metadata，然后按 image_url 下载图片。
2. 按类别抽样。例如 cpu、video-card、internal-hard-drive 各 30-60 张。
3. 用 Pillow 校验图片，统一转 RGB，并把最长边压到 768 或 1024，避免本地 embedding 太慢。
4. 保存图片、sidecar JSON 和 metadata.jsonl。
5. 下载失败或图片损坏时跳过，不中断整体流程。

7. 冒烟评测怎么做

不需要先做人工标注平台。第一版用弱监督即可：同 category 视为正例，指定 brand 时同 brand 视为更强正例。

```
{"query": "AMD Ryzen CPU", "must_category": "cpu", "must_brand": "AMD"}  
{"query": "NVIDIA RTX graphics card", "must_category": "video-card"}
```

```
{"query": "2TB SSD internal hard drive", "must_category": "internal-hard-drive"}  
{"query": "scanned email document", "must_category": "email"}
```

指标先看 category_hit@10、brand_hit@10、same_category_hit@5。只要这些冒烟指标正常，就说明图片 embedding、元信息检索、混合打分和 API 链路已经跑通。

8. 元信息怎么进入检索

硬件数据的 specs、brand、name，收据数据的 subtotal、num_items，专利数据的 caption、publication number，都统一拼成 metadata_text。检索时可以先用 CLIP 做图片/文本 embedding 召回，再用 SQLite FTS 对 metadata_text 做补充召回或 rerank。

```
final_score = 0.70 * clip_score + 0.25 * metadata_score + 0.05 * phash_score
```

这样能解决你提到的发票、专利等模板类图片问题：视觉上模板很像，CLIP 会把它们拉得很近；但金额、编号、品牌、标题、专利号等字段可以通过 FTS 或结构化过滤把结果拉开。

9. 不建议第一版做的事

- 不要默认下载 10GB 级图片数据。
- 不要把 Hugging Face loader 写进在线 API。
- 不要为了 robot 示例先处理视频数据集；如果真需要机器人图片，后续手工加 20-50 张或单独写一个小数据源即可。
- 不要要求用户申请云服务、租 GPU 或搭建外部向量数据库。
- 不要把下载后的第三方图片提交到 Git 或再分发。

10. 需要注意的限制

PCPartPicker 数据集本身是结构化 metadata，图片以 URL 方式提供；脚本要能处理 URL 失效。RVL-CDIP 小子集可能包含个人或敏感信息，因此只建议用于本地研究实验，不要公开展示样本内容。PatFig 是非商业许可，不要作为默认下载项。

11. 参考来源

- Hugging Face ImageFolder 文档: https://huggingface.co/docs/datasets/en/image_dataset
- Hugging Face Streaming 文档: <https://huggingface.co/docs/datasets/en/stream>
- PCPartPicker parts dataset: <https://huggingface.co/datasets/Doshiha/pcpartpicker-parts-dataset>
- RVL-CDIP small-200: https://huggingface.co/datasets/vaclavpechtov/rvl_cdip-small-200
- Consolidated Receipt Dataset: https://huggingface.co/datasets/Voxel51/consolidated_receipt_dataset
- PatFig: <https://huggingface.co/datasets/danaaubakirova/patfig>